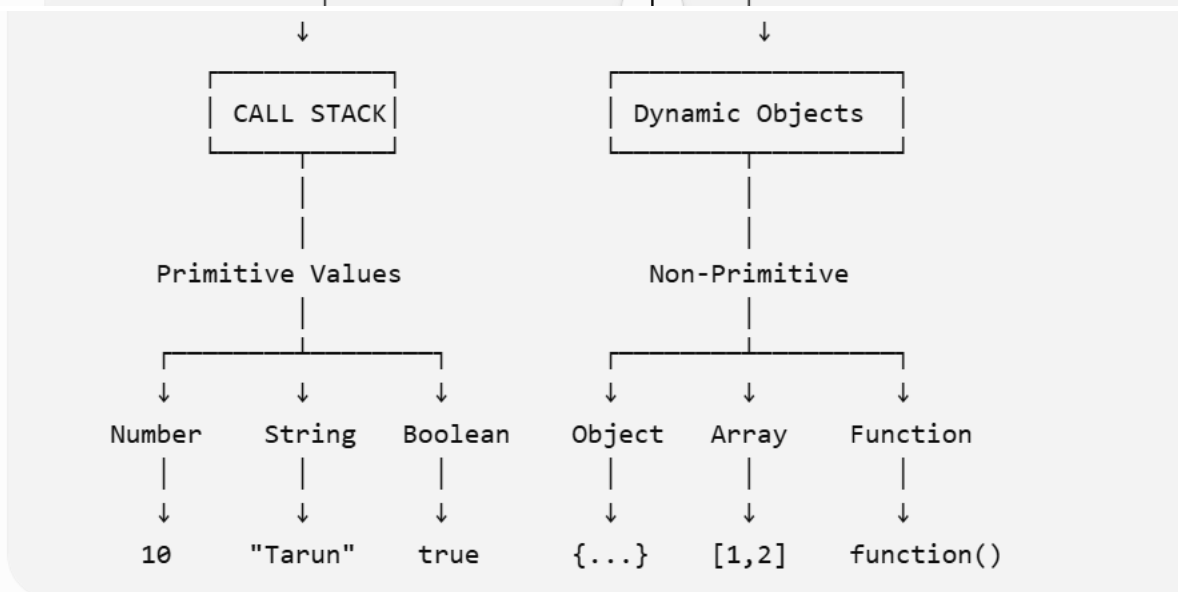
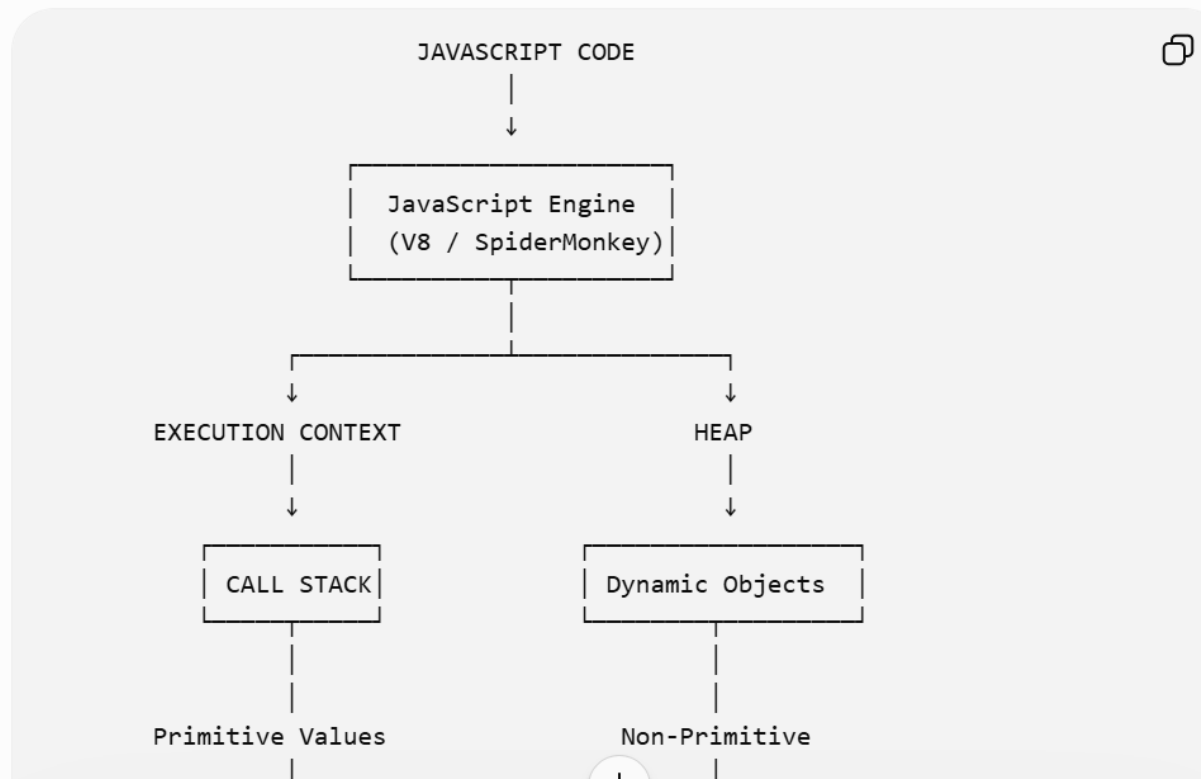
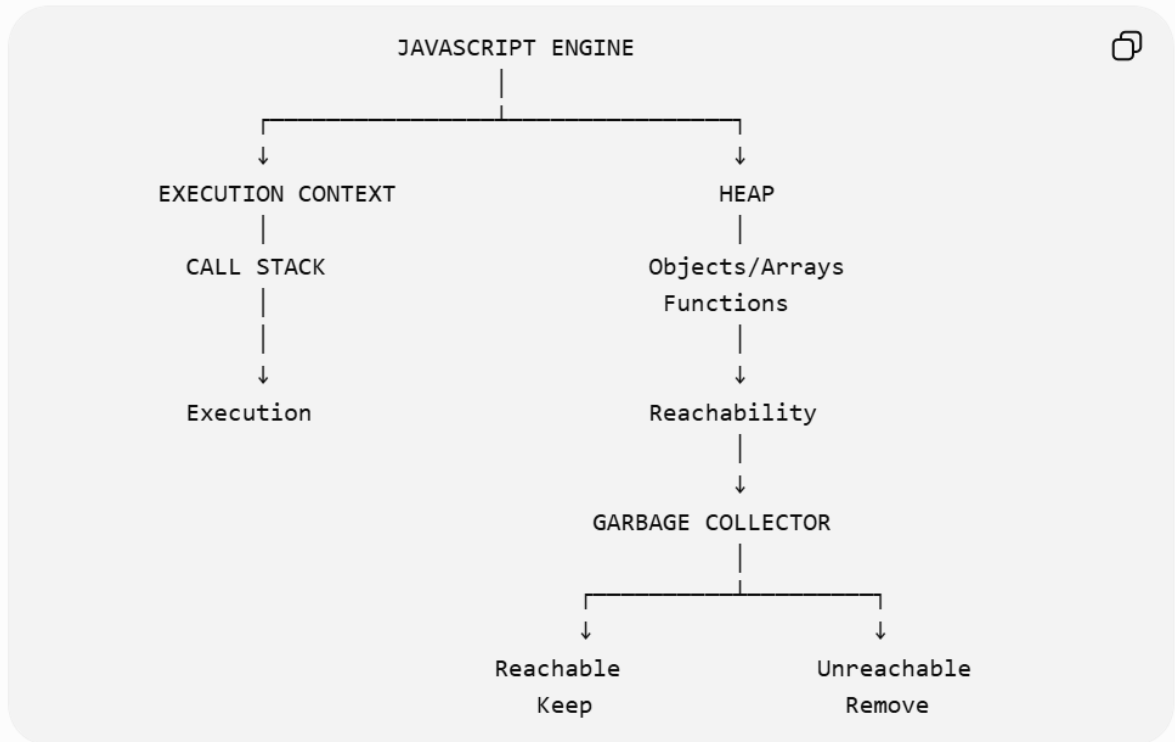


🔥 JavaScript Memory — Data Types kaha aur kaise?

✦ Up





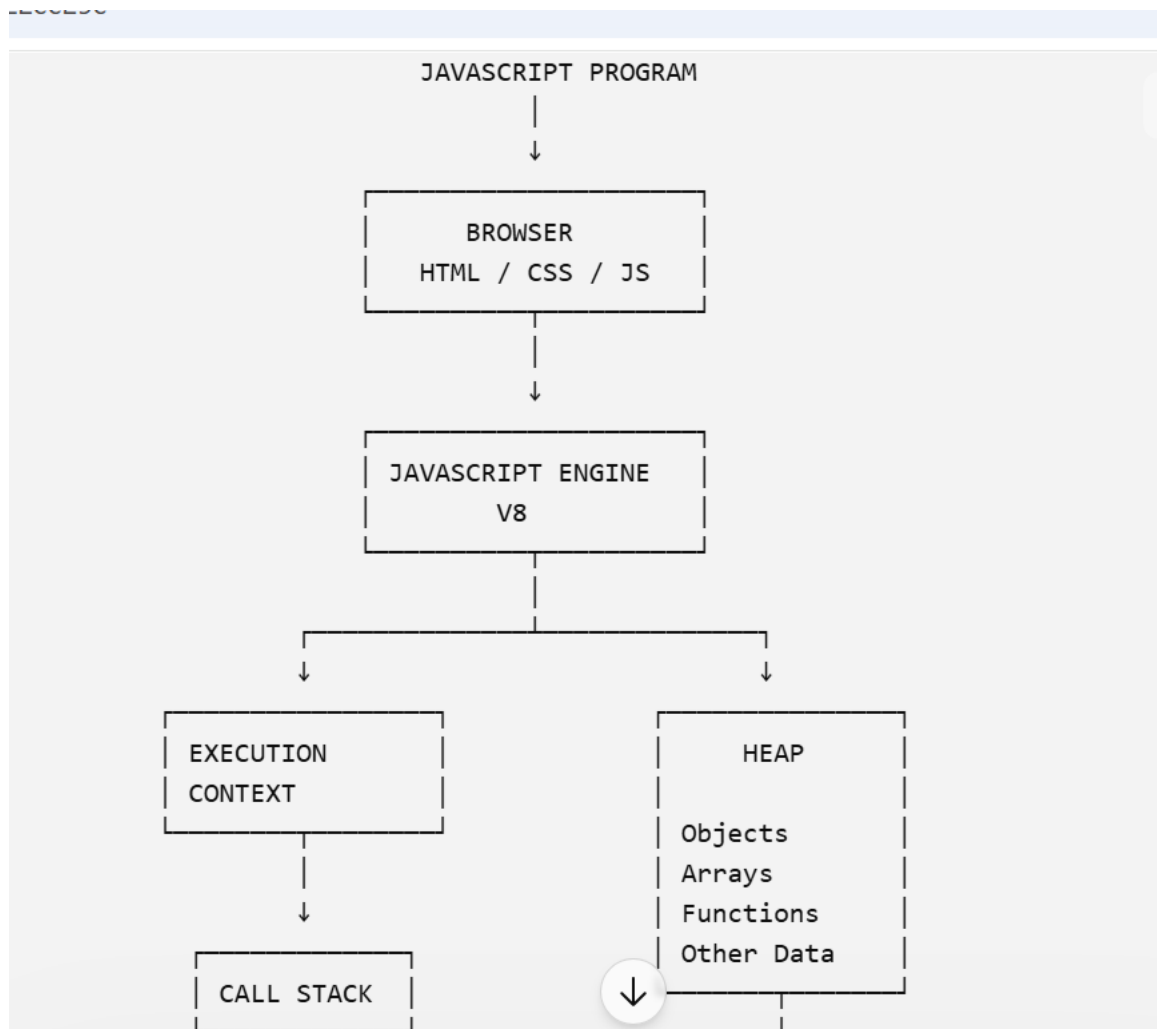
Sabse pehle ek golden rule:

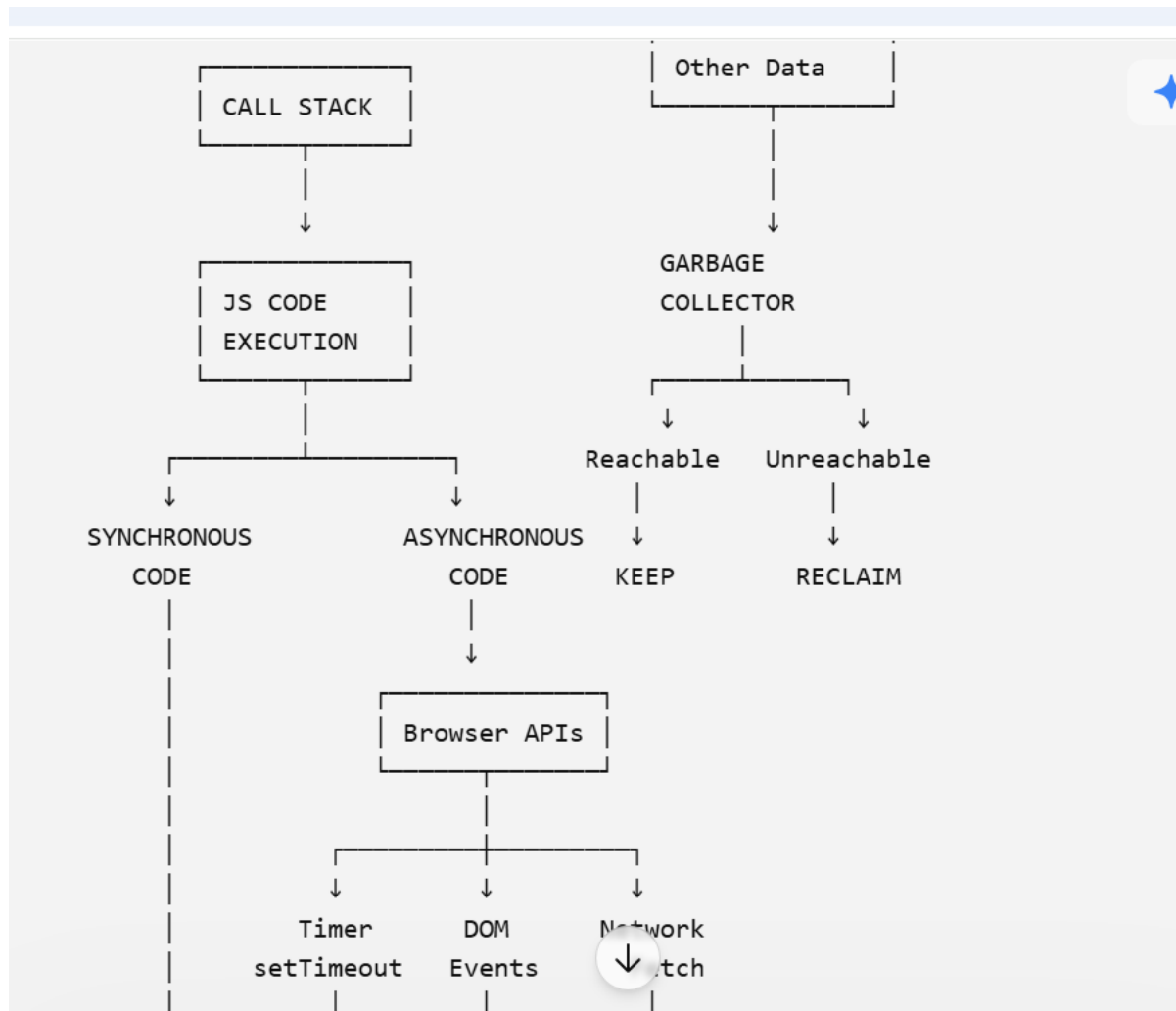
CALL STACK = JavaScript code ka execution

HEAP = objects/data ki memory

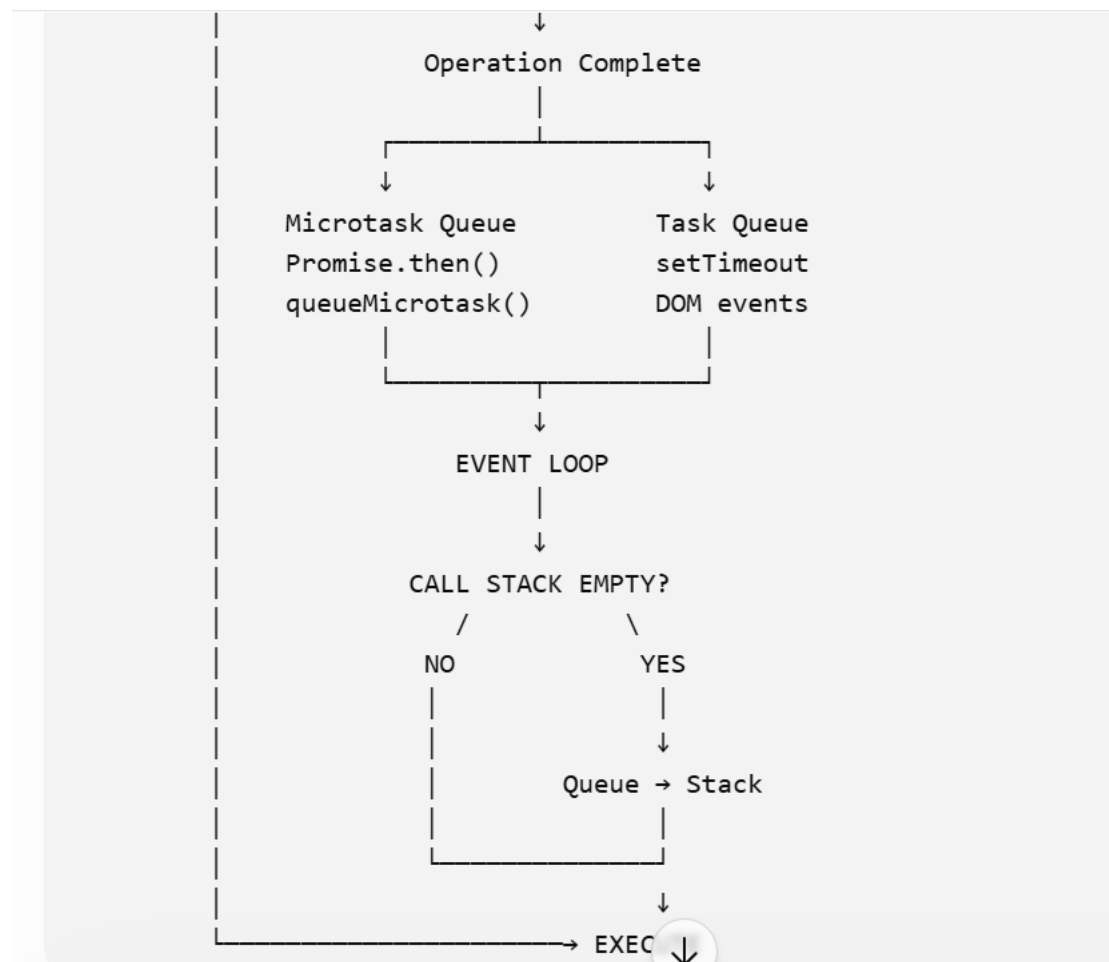
BROWSER APIs = async operations ko handle karne wali browser facilities

EVENT LOOP = queues se callback ko appropriate time par Call Stack tak pahunchane ka mechanism





00000000



1. Prototype kya hota hai?

✦ Upgrade

Simple definition

Prototype JavaScript ka ek object hota hai jiske andar properties aur methods rakhe ja sakte hain, aur doosre objects usse properties/methods inherit kar sakte hain.

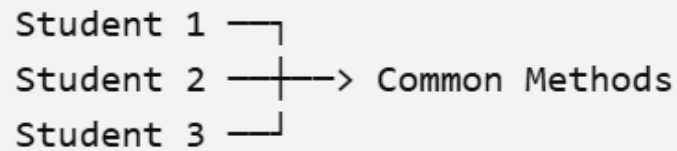
Simple language me:

Prototype = ek parent-like object jahan common properties/methods rakhe ja sakte hain, jinko child objects use kar sakte hain.

JavaScript me inheritance ka main mechanism **prototype chain** hai.

2. Real-life example

Maan lo college me:



Har student ke paas:

```
name  
age  
rollNo
```

alag-alag ho sakte hain.

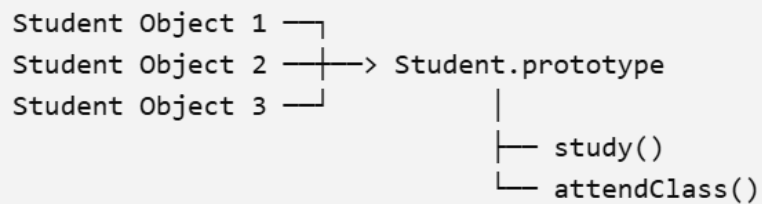
Lekin:

```
study()  
attendClass()
```

sabke liye same hai.

sabke liye same hai.

Toh hum `study()` aur `attendClass()` ko har object ke andar alag-alag banane ki jagah prototype me rakh sakte hain.



Isse memory efficiently use hoti hai.

3. Object ke andar prototype kaise aata hai?

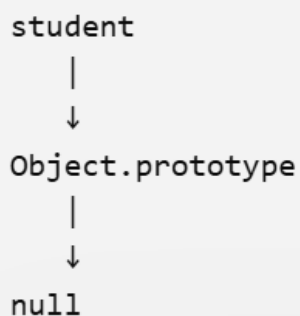
Example:

JavaScript

```
let student = {
  name: "Tarun",
  age: 22
};
```

JavaScript internally is object ko ek prototype se connect karta hai.

Conceptually:



`Object.prototype` me bahut common methods hote hain:

`</> JavaScript`

```
toString()
valueOf()
hasOwnProperty()
isPrototypeOf()
```

Isliye tum:

`</> JavaScript`

```
student.toString();
```

kar sakte ho, even though tumne `toString()` khud `student` ke andar nahi likha.

rr

Tumhare 4 methods ko ek table mein dekho

Method	Kaam	Example	Result
<code>toString()</code>	Object ko string representation deta hai	<code>a.toString()</code>	<code>[object Object]</code>
<code>valueOf()</code>	Object ki value return karta hai	<code>a.valueOf()</code>	<code>a</code> object
<code>hasOwnProperty()</code>	Check karta hai property directly object mein hai ya nahi	<code>a.hasOwnProperty("name")</code>	<code>true</code>
<code>isPrototypeOf()</code>	Check karta hai ki kya ye object doosre object ka prototype hai	<code>a.isPrototypeOf(b)</code>	<code>true/false</code>

4. Prototype chain kya hoti hai?



Ye bahut important interview concept hai.

Jab JavaScript kisi object ki property/method search karta hai, toh pehle object ke andar dekhta hai.

Agar nahi mili:

```
Object
↓
Prototype
↓
Prototype ka Prototype
↓
...
↓
null
```



Is process ko **Prototype Chain** kehte hain.

```
</> JavaScript  
  
let student = {  
  name: "Tarun"  
};  
  
console.log(student.toString());
```

JavaScript kya karega?

Step 1

Pehle:

```
</> JavaScript  
  
student
```

me `toString` search karega.

Nahi mila.

Step 2

`student` ke prototype me search karega:

```
</> JavaScript  
Object.prototype
```

Yahan:

```
</> JavaScript  
toString()
```

mil jayega.

Isliye code work karega.

```
javascript

const a = {
  name: "Tarun"
};

const b = {
  marks: 54
};

const c = {
  age: 22
};

const d = {
  city: "Indore"
};

// Prototype chain banana
b.__proto__ = a;
c.__proto__ = b;
d.__proto__ = c;
```

JavaScript

```
// Property access
console.log(d.city);    // Indore
console.log(d.age);     // 22
console.log(d.marks);   // 54
console.log(d.name);    // Tarun
```

Ab chain dekho

Humne ye banaya:

```
d
↓
c
↓
b
↓
a
↓
Object.prototype
↓
null
```

5. `__proto__` kya hai?



Ab sabse important confusion.

Definition

`__proto__` ek accessor property hai jo kisi object ke internal prototype ko access/set karne ke liye use hoti hai.

Example:

`</> JavaScript`

```
let student = {  
  name: "Tarun"  
};  
  
console.log(student.__proto__);
```

Output conceptually:

Object.prototype



6. `prototype` vs `__proto__`

[★ Upgrade](#)

Ye interview me bahut poocha ja sakta hai.

Point	<code>prototype</code>	<code>__proto__</code>
Kya hai?	Ek object/property	Prototype ko access karne wala accessor
Mainly kahan milta hai?	Constructor functions/classes ke saath	Objects par
Purpose	Future objects ke liye shared properties/methods	Kisi object ka prototype access karna
Example	<code>Student.prototype</code>	<code>student.__proto__</code>
Relation	Constructor ka prototype object	Object ka actual prototype
Common use	Methods define karna	Prototype chain dekhna
Recommended?	Yes	↓ Direct use generally avoid

7. Constructor Function kya hota hai?

Prototype ko samajhne ke liye constructor function samjho.

```
</> JavaScript

function Student(name, age) {
  this.name = name;
  this.age = age;
}
```

Ab:

```
</> JavaScript

let s1 = new Student("Tarun", 22);
let s2 = new Student("Rahul", 21);
```

Yahan:

```
function Student(name, marks) {  
    this.name = name;  
    this.marks = marks;  
  
    this.printMarks = function () {  
        console.log("Marks =", this.marks);  
    };  
}  
  
const s1 = new Student("Tarun", 54.4);  
const s2 = new Student("Rahul", 80);  
  
console.log(s1.name); // Tarun  
console.log(s1.marks); // 54.4  
  
s1.printMarks(); // Marks = 54.4  
  
console.log(s2.name); // Rahul  
console.log(s2.marks); // 80  
  
s2.printMarks(); // Marks = 80
```

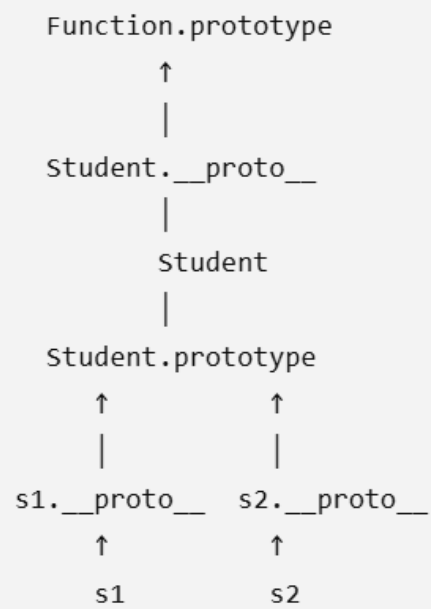


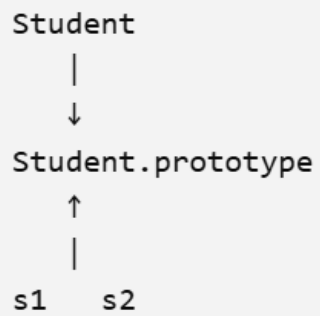
```

console.log("====="); // =====

console.log(s1.__proto__ === Student.__proto__); // false
console.log(s1.__proto__ === s2.__proto__); // true
console.log(s1.__proto__ === Student.prototype); // true
console.log(Student.__proto__ === Function.prototype); // true

```





Dono objects ka prototype:

```
</> JavaScript  
Student.prototype
```

hoga.

Check:

```
</> JavaScript  
console.log(s1.__proto__ === Student.prototype);
```

ee

8. User-defined Prototype kya hota hai?

[Upgra](#)

Agar hum khud kisi constructor ke prototype me method/property add karte hain, toh use commonly **user-defined prototype methods/properties** bol sakte hain.

Example:

</> JavaScript

```
function Student(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
Student.prototype.study = function() {  
  console.log(this.name + " is studying");  
};
```

Ab:

</> JavaScript

```
let s1 = new Student("Tarun", 22);  
let s2 = new Student("Rahul", 21);
```

Conceptually:

```
s1
├─ name: "Tarun"
└─ age: 22
```

`study()` directly nahi hai.

Lekin:

```
s1
↓
Student.prototype
↓
study()
```

Isliye:

```
</> JavaScript

s1.study();
```

ff

```
function Student(name, marks, age) {  
    this.name = name;  
    this.marks = marks;  
    this.age = age;  
}  
  
// User-defined prototype method 1  
Student.prototype.study = function () {  
    console.log(this.name + " is studying");  
};  
  
// User-defined prototype method 2  
Object.defineProperty(Student.prototype, "printMarks", {  
    value: function () {  
        console.log("Marks =", this.marks);  
    },  
    writable: true,  
    configurable: true  
});
```



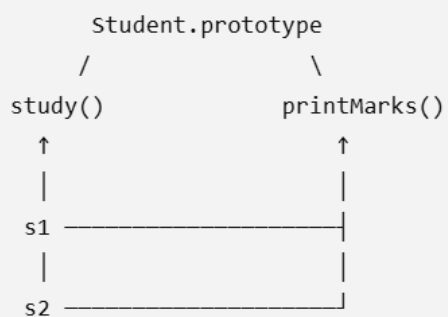
```
// Objects create
const s1 = new Student("Tarun", 54.4, 22);
const s2 = new Student("Rahul", 80, 21);

// Prototype methods call
s1.study();           // Tarun is studying
s1.printMarks();      // Marks = 54.4

s2.study();           // Rahul is studying
s2.printMarks();      // Marks = 80

// Prototype checking
console.log(s1.__proto__ === Student.prototype); // true
console.log(s2.__proto__ === Student.prototype); // true
console.log(s1.study === s2.study);              // true
console.log(s1.printMarks === s2.printMarks);     // true
```

Iska prototype structure



Yani `study()` aur `printMarks()` har object ke andar separately nahi hain. Dono `Student.prototype` mein hain, aur `s1 / s2` prototype chain ke through unhe access karte hain.

10. Ye memory efficient kyun hai?

Agar hum method constructor ke andar likhen:

</> JavaScript

```
function Student(name, age) {  
  
    this.name = name;  
    this.age = age;  
  
    this.study = function() {  
        console.log("Studying");  
    };  
}
```

Agar:

</> JavaScript

```
let s1 = new Student("Tarun", 22);  
let s2 = new Student("Rahul", 21);  
let s3 = new Student("Amit", 20);
```



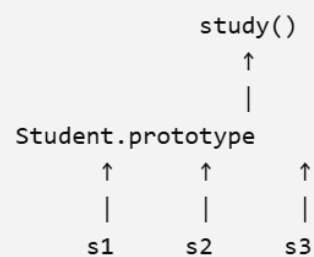
Toh conceptually har object ke liye separate function ban sakta hai:

```
s1 → study()  
s2 → study()  
s3 → study()
```

Lekin prototype me:

```
</> JavaScript  
  
Student.prototype.study = function() {  
  console.log("Studying");  
};
```

Toh:



Ek shared method use hota hai.

Normal object:

```
</> JavaScript  
  
let obj = {};
```

Is object ke paas normally:

```
</> JavaScript  
  
obj.prototype
```

nahi hota.

Function:

```
</> JavaScript  
  
function Student() {}
```

```
</> JavaScript
```

```
function Student() {}
```

Function ke paas:

```
</> JavaScript
```

```
Student.prototype
```

hota hai.

Check:

```
</> JavaScript
```

```
console.log(Student.prototype);
```

12. Function ke paas `prototype` kyun hota hai?

Because function ko constructor ki tarah use kiya ja sakta hai:

```
</> JavaScript
function Student(name) {
  this.name = name;
}
```

Jab:

```
</> JavaScript
new Student("Tarun");
```

likhte ho, JavaScript naye object ko approximately is prototype se connect karta hai:

```
Student.prototype
```

Conceptually `new` ke important steps:



likhte ho, JavaScript naye object ko approximately is prototype se connect karta hai:

```
Student.prototype
```



Conceptually `new` ke important steps:

```
new Student("Tarun")
↓
New empty object create
↓
Object ka prototype = Student.prototype
↓
Student function this ke saath execute
↓
New object return
```



-29a6c22ee25e

13. new aur prototype ka connection

Example:

```
</> JavaScript

function Student(name) {
  this.name = name;
}

let s1 = new Student("Tarun");
```

Conceptually:

```
graph TD
    s1 --> name["name = 'Tarun'"]
    s1 --> prototype["[[Prototype]]"]
    prototype --> Student_prototype["Student.prototype"]
```

14. `[[Prototype]]` kya hai?

Ye bhi important hai.

JavaScript objects ke andar ek **internal slot** hota hai:

```
[[Prototype]]
```

Ye actual internal prototype relationship ko represent karta hai.

Tum directly normally:

```
</> JavaScript  
obj.[[Prototype]]
```

nahi likh sakte.

Access karne ke common ways:

Access karne ke common ways:

```
</> JavaScript  
  
obj.__proto__
```

ya recommended:

```
</> JavaScript  
  
Object.getPrototypeOf(obj)
```

Example:

```
</> JavaScript  
  
let s1 = new Student("Tarun");  
  
console.log(Object.getPrototypeOf(s1));
```


15. `__proto__` aur `[[Prototype]]` same hain?

Conceptually:

```
[[Prototype]]
  ↑
  |
internal mechanism

__proto__
  ↑
  |
accessor through which you can access it
```

So:

`[[Prototype]]` = internal mechanism/slot

`__proto__` = accessor

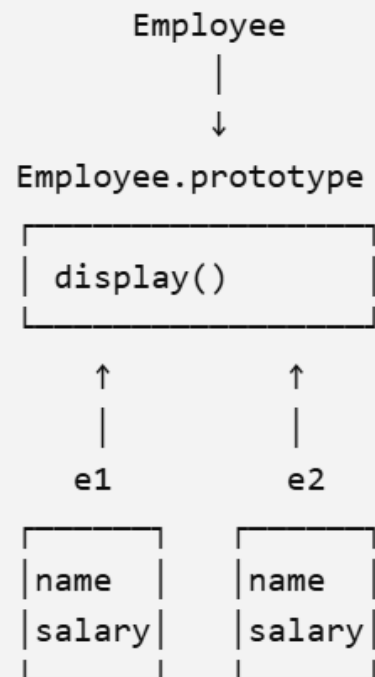
Interview me ye distinction bolna impressive hota hai.

16. User-defined prototype ka proper example

</> JavaScript

```
function Employee(name, salary) {  
    this.name = name;  
    this.salary = salary;  
}  
  
Employee.prototype.display = function() {  
    console.log(this.name + " earns " + this.salary);  
};  
  
let e1 = new Employee("Tarun", 50000);  
let e2 = new Employee("Rahul", 60000);  
  
e1.display();  
e2.display();
```

Structure:



`display()` ek hi jagah defined hai.

17. Prototype me property bhi bana sakte hain

Sirf method hi nahi.

```
</> JavaScript

function Student(name) {
  this.name = name;
}

Student.prototype.college = "Medi-Caps";

let s1 = new Student("Tarun");
let s2 = new Student("Rahul");

console.log(s1.college);
console.log(s2.college);
```

Output:

Medi-Caps
Medi-Caps



20. Prototype Chain ka complete example

</> JavaScript

```
function Student(name) {  
    this.name = name;  
}  
  
Student.prototype.study = function() {  
    console.log("Studying");  
};  
  
let s1 = new Student("Tarun");
```

Chain approximately:

```
s1  
|  
| [[Prototype]]  
↓  
Student.prototype  
|  
| [[Prototype]]
```



Chain approximately:

```
s1
|
| [[Prototype]]
↓
Student.prototype
|
| [[Prototype]]
↓
Object.prototype
|
| [[Prototype]]
↓
null
```

Isko hi:

`__proto__` is an accessor property that provides access to an object's internal `[[Prototype]]`.

`[[Prototype]]`

`[[Prototype]]` is an internal object reference used by JavaScript to establish the prototype chain and perform property lookup.

Prototype Chain

Prototype chain is the sequence of objects JavaScript follows while looking for a property or method, ending at `null`.

User-defined Prototype

When a developer adds custom properties or methods to a constructor's prototype object, those are user-defined prototype properties or methods.

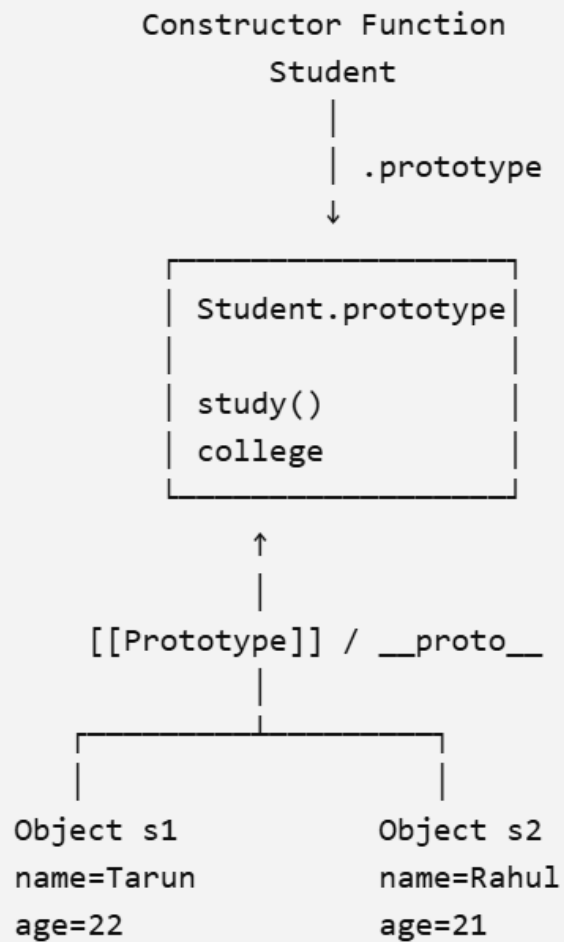
```
</> JavaScript
Student.prototype.study = function() {
  console.log("Studying");
};
```

★ Upgr



29. Sabko ek diagram me yaad karo

29. Sabko ek diagram me yaad karo



Aur prototype ki bhi ek prototype hoti hai:



Bilkul bhai. JavaScript `class` me `prototype` ko samajhne ke liye ek baat pehle clear kar lo:

JavaScript ki `class` syntax modern/easy syntax hai, lekin internally JavaScript inheritance ka mechanism prototype-based hi hai.

Matlab `class` alag se koi completely new inheritance system nahi hai. Class ke methods actually prototype object par store hote hain.

1. JavaScript Class kya hoti hai?



Definition

Class JavaScript me objects create karne ka ek blueprint/template hoti hai.

Example:

```
</> JavaScript

class Student {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  study() {
    console.log(this.name + " is studying");
  }
}
```

Ab objects:

```
</> JavaScript

class Student {
  constructor(name) {
    this.name = name;
  }

  study() {
    console.log("Studying");
  }
}
```

likhte ho, JavaScript conceptually:

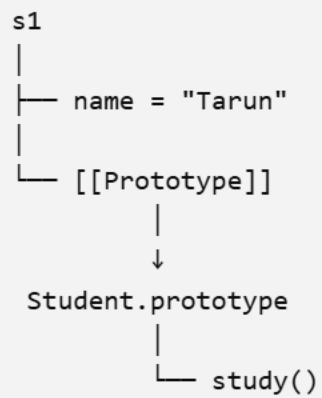
```
Student
  |
  ↓
Student.prototype
  |
  └─ study()
```

3. Object banane ke baad structure

JavaScript

```
let s1 = new Student("Tarun");
```

Conceptually structure:



lclive

5. Class ka method prototype me hota hai

Example:

</> JavaScript

```
class Student {  
  
  constructor(name) {  
    this.name = name;  
  }  
  
  study() {  
    console.log("Studying");  
  }  
  
  sleep() {  
    console.log("Sleeping");  
  }  
}
```

Conceptually:

Conceptually:

Student.prototype

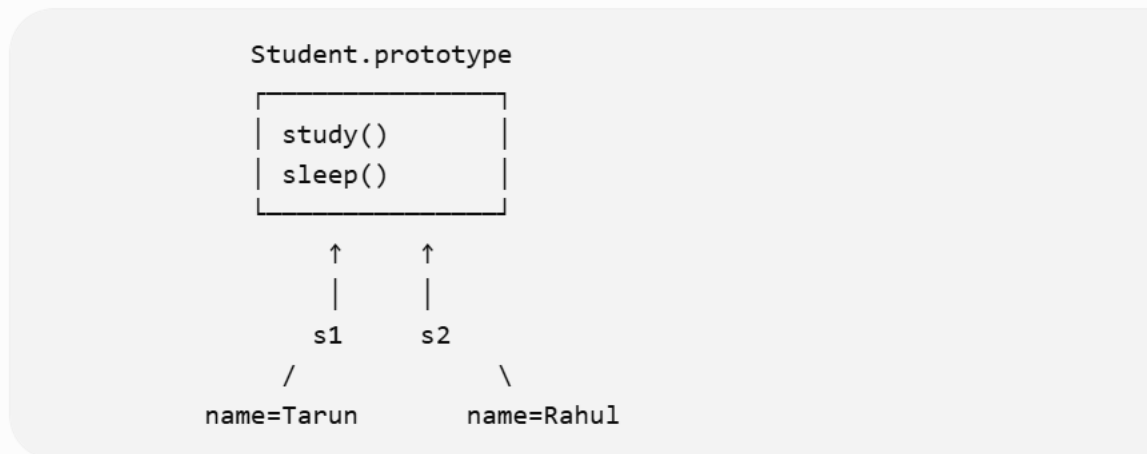
```
|  
├─ study()  
└─ sleep()
```

Aur:

</> JavaScript

```
let s1 = new Student("Tarun");  
let s2 = new Student("Rahul");
```

Structure:



Ek hi `study()` method ko multiple objects share kar rahe hain.

`</>` JavaScript

```
class A {
  #marks;
  name = "tarun";
  #salary = 5444.44;

  setmarks(marks) {
    this.#marks = marks;
  }

  getmarks() {
    return this.#marks;
  }

  rectangle(length, width) {
    let area = length * width;
    console.log(area);
  }

  greet() {
    console.log("hello", this.name);
  }
}
```



```
// JavaScript
```

```
    getsalary() {  
        return this.#salary;  
    }  
}
```

```
const obj = new A();
```

```
obj.setmarks(443.3);
```

```
console.log(obj.getmarks());
```

```
obj.rectangle(10, 20);
```

```
obj.greet();
```

```
console.log(obj.getsalary());
```

1. Sabse pehle: `__proto__` kya hai?

`__proto__` kisi object ke internal `[[Prototype]]` ko access karta hai.

Example:

```
</> JavaScript

class Student {
  study() {
    console.log("Studying");
  }
}

let s1 = new Student();
```

Ab:

```
</> JavaScript

console.log(s1.__proto__);
```

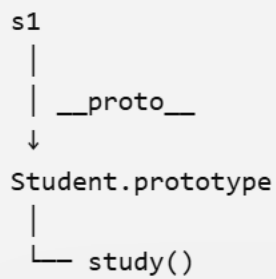
2. Class ke object ka `__proto__`

```
</> JavaScript

class Student {
  study() {
    console.log("Studying");
  }
}

let s1 = new Student();
```

Structure:



```
s1 me study()?
↓
No
↓
s1.__proto__ = Student.prototype
↓
Student.prototype me study()?
↓
Yes
↓
Execute
```


3. Lekin `Student.__proto__` kya hai?

[✦ Upgrade](#)

Ab dhyan dena:

```
</> JavaScript
```



```
Student.__proto__
```

ye `s1.__proto__` se different hai.

`Student` khud ek **function object/class object** hai.

JavaScript classes internally function-like objects hain, aur class constructor ka prototype chain `Function.prototype` tak jaata hai.

```
Student
  |
  | __proto__
  ↓
Function.prototype
  |
  ↓
Object.prototype
  |
  ↓
null
```

Check:

```
</> JavaScript
```

```
console.log(Student.__proto__ === Function.prototype);
```

Output:

```
true
```



Js oops

Code	Kis ka <code>__proto__</code> / prototype?	Result	Yaad rakho	
<code>s1.__proto__</code>	<code>s1</code> ka prototype	<code>Student.prototype</code>	Object ka prototype	
<code>Student.prototype</code>	<code>Student</code> instances ke liye prototype object	<code>study()</code> wala object	<code>s1</code> isi ko point karta hai	
<code>Student.__proto__</code>	<code>Student</code> class/function object ka prototype	<code>Function.prototype</code>	Class khud function-like object hai	
<code>Object.getPrototypeOf(s1)</code>	<code>s1</code> ka prototype	<code>Student.prototype</code>	<code>s1.__proto__</code> ka modern way	
<code>Object.getPrototypeOf(Student)</code>	<code>Student</code> ka prototype	<code>Function.prototype</code>	<code>Student.__proto__</code> ka equivalent	

`</>` JavaScript

```
s1.__proto__ === Student.prototype
```

 true

Lekin:

`</>` JavaScript

```
Student.__proto__ === Student.prototype
```

 false

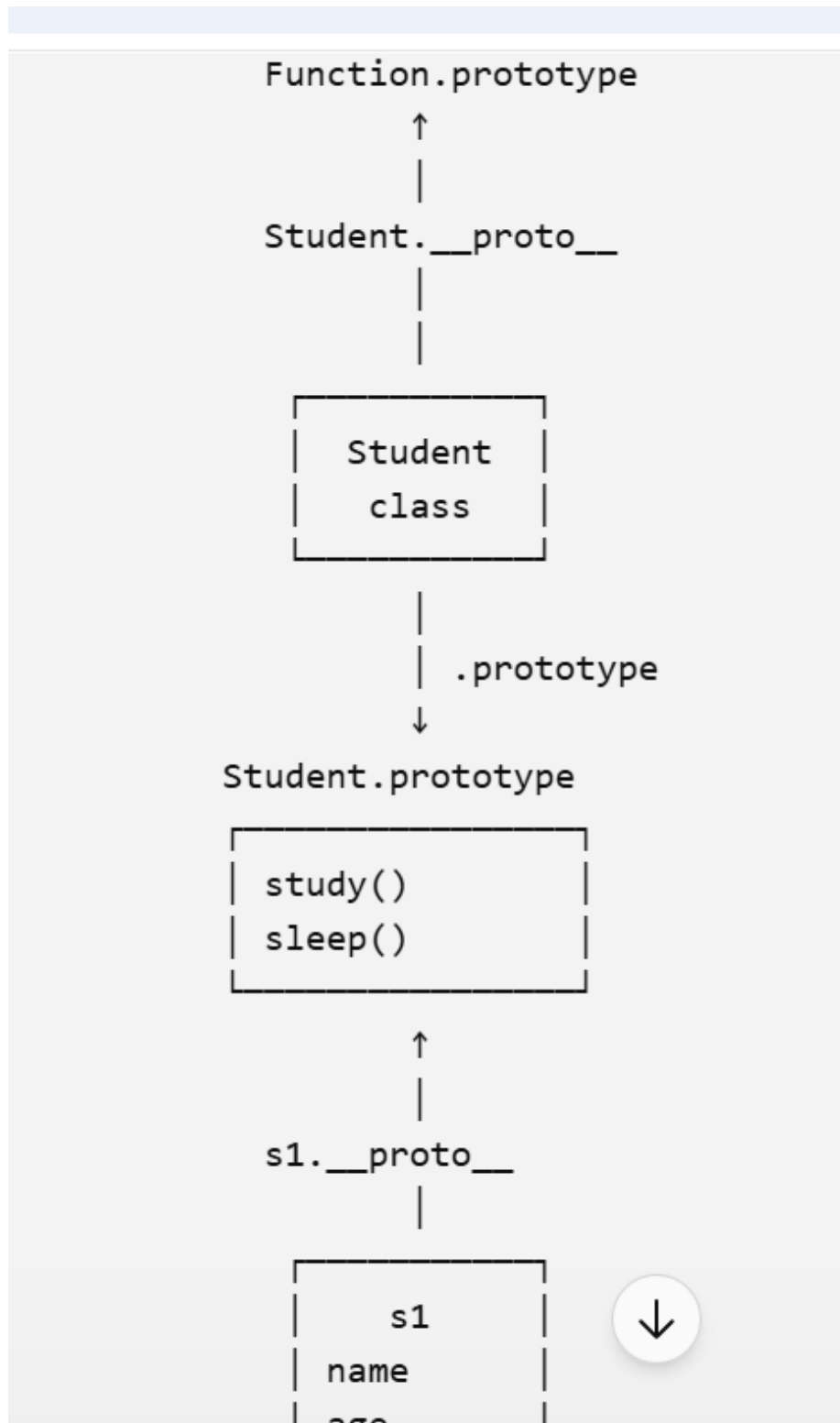
Instead:

`</>` JavaScript

```
Student.__proto__ === Function.prototype
```

 true





1. User-defined Prototype kya hota hai?



Full definition

Jab programmer JavaScript me kisi constructor function ya class ke `prototype` object par apni custom properties ya methods define karta hai, unhe user-defined prototype properties/methods kaha ja sakta hai.

Example:

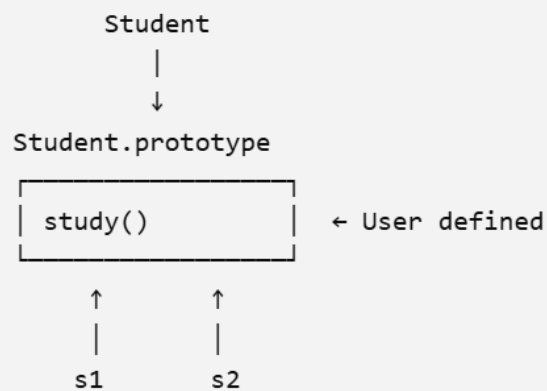
```
</> JavaScript   
  
function Student(name) {  
    this.name = name;  
}  
  
Student.prototype.study = function() {  
    console.log(this.name + " is studying");  
};
```

```
</> JavaScript
```

```
Student.prototype.study
```

humne khud define kiya hai, isliye ye user-defined prototype method hai.

2. Iska structure dekho



Objects



4. Yahan `__proto__` ka role

</> JavaScript

```
console.log(s1.__proto__);
```

Ye:

</> JavaScript

```
Student.prototype
```

ko point karega.

Check:

</> JavaScript

```
console.log(s1.__proto__ === Student.prototype);
```

Output:



true

8. User-defined prototype ka main benefit



Suppose 1000 students hain.

Agar method constructor ke andar likho:

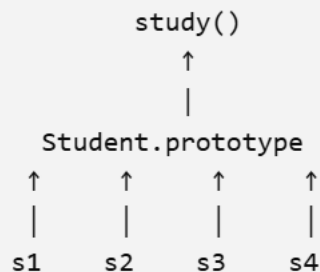
```
</> JavaScript  
  
function Student(name) {  
    this.name = name;  
  
    this.study = function() {  
        console.log("Studying");  
    };  
}
```

Toh methods unnecessarily har object ke liye create ho sakte hain.

Instead:

```
</> JavaScript  
  
Student.prototype.study = function() {  
    console.log("Studying");  
};
```

Toh:



Ek shared method.

f3-29a6c22ee25e

9. Prototype chain bhi samjho

Agar:

```
</> JavaScript
```

```
function Student(name) {  
    this.name = name;  
}  
  
Student.prototype.study = function() {  
    console.log("Studying");  
};  
  
let s1 = new Student("Tarun");
```

Toh chain:

```
s1  
↓  
Student.prototype  
↓  
Object.prototype  
↓  
...
```



10. User-defined prototype manually `__proto__` se bhi set kar sakte ho

Example:

```
</> JavaScript

let person = {
  greet: function() {
    console.log("Hello");
  }
};

let student = {
  name: "Tarun"
};

student.__proto__ = person;
```

Ab:



Ab:

```
</> JavaScript

student.greet();
```

Output:

Hello

Structure:

```
student
  ↓ __proto__
person
  ↓
greet()
```

11. Object.create() aur user-defined prototype

Ye aur clean example hai:

```
</> JavaScript

let person = {
  greet() {
    console.log("Hello");
  }
};

let student = Object.create(person);

student.name = "Tarun";
```

Ab:

```
</> JavaScript

student.greet();
```



work karega.

</> JavaScript

```
function Employee(name, salary) {  
    this.name = name;  
    this.salary = salary;  
}  
  
Employee.prototype.display = function() {  
    console.log(  
        this.name + " earns " + this.salary  
    );  
};  
  
Employee.prototype.company = "ABC Pvt Ltd";  
  
let e1 = new Employee("Tarun", 50000);  
let e2 = new Employee("Rahul", 60000);  
  
e1.display();  
e2.display();  
  
console.log(e1.company);  
console.log(e2.company);  
  
console.log(e1.__proto__ === Employee.prototype);
```



```
class A {  
  
    #A_ka_private = "tarun";  
  
    A_ka_public = 55;  
  
    showA() {  
        console.log("A Private:", this.#A_ka_private);  
        console.log("A Public:", this.A_ka_public);  
    }  
}  
  
class B extends A {  
  
    #B_ka_private = 44.33;  
  
    B_ka_public = "a";  
  
    showB() {  
        // A ka private ❌  
        // console.log(this.#A_ka_private);  
    }  
}
```



</> JavaScript

```
showB() {  
  // A ka private ❌  
  // console.log(this.#A_ka_private);  
  
  // A ka public ✅  
  console.log("A Public:", this.A_ka_public);  
  
  // B ka private ✅  
  console.log("B Private:", this.#B_ka_private);  
  
  // B ka public ✅  
  console.log("B Public:", this.B_ka_public);  
}  
}  
  
let b = new B();  
  
// B object se  
console.log(b.A_ka_public); // ✅  
console.log(b.B_ka_public); // ✅  
↓
```

</> JavaScript

// B object se

console.log(b.A_ka_public); // ✓

console.log(b.B_ka_public); // ✓

// Private direct access

// console.log(b.#A_ka_private); // ✗

// console.log(b.#B_ka_private); // ✗

// A ke method ke through A private

b.showA();

// B ke method ke through

b.showB();

// C++ pointer ka closest reference-like example

let ptr = b;

console.log(ptr.A_ka_public); // ✓

// console.log(ptr.#A_ka_private); // ✗



11. Access table

Member	b object se	B class ke andar	A class ke andar
#A_ka_private	✗	✗	✓
A_ka_public	✓	✓	✓
#B_ka_private	✗	✓	✗
B_ka_public	✓	✓	✓

```
class A {  
  
    #A_ka_private(a, b) {  
        console.log("A ka private function:", a + b);  
    }  
  
    callAPrivate(a, b) {  
        this.#A_ka_private(a, b);  
    }  
  
    A_ka_public(name, age, ispass) {  
        console.log(name, age, ispass);  
    }  
}  
  
class B extends A {  
  
    #B_ka_private(a, b) {  
        console.log("Decimal power:", a ** b);  
    }  
}
```

```

class B extends A {

    #B_ka_private(a, b) {
        console.log("Decimal power:", a ** b);
    }

    callBPrivate(a, b) {
        this.#B_ka_private(a, b);
    }

    B_ka_public(a, b, c) {
        console.log(a * b * c);
    }
}

let b = new B();

b.callAPrivate(10, 20); // A ka private method execute
b.callBPrivate(2, 3);  // B ka private method execute

```



JavaScript me constructor ko `#constructor()` karke private nahi bana sakte.

Constructor method ka naam hamesha `constructor()` hi hota hai.

Constructor ki accessibility C++ ke `public/private/protected constructor` jaisi directly define nahi hoti.

1. A ka public constructor + B ka public constructor

</> JavaScript

```
class A {  
  
    constructor(name, age) {  
        console.log("A ka constructor");  
        console.log(name, age);  
    }  
  
    A_ka_public() {  
        console.log("A ka public method");  
    }  
}  
  
class B extends A {  
  
    constructor(name, age, marks) {  
        super(name, age);  
  
        console.log("B ka constructor");  
        console.log(marks);  
    }  
}
```

```
</> JavaScript
```

```
    B_ka_public() {  
        console.log("B ka public method");  
    }  
}  
  
let b = new B("Tarun", 22, 85);
```

Output:

```
A ka constructor  
Tarun 22  
  
B ka constructor  
85
```

29a6c22ee25e

Flow:

```
new B()  
  ↓  
B constructor  
  ↓  
super(name, age)  
  ↓  
A constructor  
  ↓  
B constructor continues
```

2. `super()` kyu?

Jab B `extends` A karta hai aur B ka apna constructor hai:

```
</> JavaScript

class B extends A {
  constructor() {
    super();
  }
}
```

to `super()` parent class A ke constructor ko call karta hai.

3. Kya A ka constructor B ke object se call kar sakte hain?

Haan, indirectly.

```
</> JavaScript

let b = new B("Tarun", 22, 85);
```

B constructor ke andar:

```
</> JavaScript

super("Tarun", 22);
```

A constructor execute karta hai.

```
b = new B()
  ↓
B constructor
  ↓
super()
  ↓
```

1. `super` kya hai?

Definition

`super` ek special keyword hai jo child class (`B`) ke andar parent class (`A`) ke constructor aur methods ko access/call karne ke liye use hota hai.

`super` mainly 2 jagah use hota hai:

```
super()  
super.method()
```

2. `super()` — Parent constructor call

Example:

```
</> JavaScript  
  
class A {  
    constructor(name, age) {  
        console.log("A constructor");  
        console.log(name, age);  
    }  
}  
  
class B extends A {  
    constructor(name, age, marks) {  
        super(name, age);  
        console.log("B constructor");  
        console.log(marks);  
    }  
}
```



`</> JavaScript`

```
let b = new B("Tarun", 22, 85);
```

Output:

```
A constructor  
Tarun 22  
B constructor  
85
```

Flow:

```
new B("Tarun",22,85)  
  ↓  
B constructor  
  ↓  
super("Tarun",22)  
  ↓  
A constructor  
  ↓  
B constructor continues
```



4. `super.method()` — Parent method call

Ab constructor ke alawa **parent ka method** bhi call kar sakte ho.

</> JavaScript

```
class A {
    show() {
        console.log("A ka show method");
    }
}

class B extends A {
    show() {
        console.log("B ka show method");
        super.show();
    }
}

let b = new B();
```



7. `super()` aur `super.method()` difference

Syntax	Meaning
<code>super()</code>	Parent constructor call
<code>super.method()</code>	Parent method call
<code>this</code>	Current object
<code>super.method()</code>	Parent implementation

```
class A {  
  
    constructor(name, age) {  
        this.name = name;  
        this.age = age;  
  
        console.log("A constructor");  
    }  
  
    show() {  
        console.log("A show");  
    }  
}  
  
class B extends A {  
  
    constructor(name, age, marks) {  
  
        super(name, age);  
  
        this.marks = marks;  
    }  
}
```



```
        this.marks = marks;

        console.log("B constructor");
    }

    show() {

        console.log("B show");

        super.show();
    }
}

let b = new B("Tarun", 22, 85);

b.show();
```

Output:

```
A constructor
B constructor
B show
```



Js oops

#	Point	this	super	Upgrade	Sh
1	Meaning	Current object ko refer karta hai	Parent class ke members ko refer karta hai		
2	Mainly used	Current class/object ke data aur methods ke liye	Parent class ke constructor/methods ke liye		
3	Constructor	Current object ki properties initialize karta hai	Parent constructor ko call karta hai		
4	Syntax	this.name	super(name)		
5	Parent method	Current/child method ko access karta hai	Parent method ko specifically call karta hai		
6	Property	this.age	super.age possible in appropriate contexts		
7	Method	this.show()	super.show()		
8	Constructor me	super() ke baad derived	Parent constructor ko initialize		

-29a6c22ee25e

7	Method	this.show()	super.show()	Upgrade	
8	Constructor me	super() ke baad derived class me use karna hota hai	Parent constructor ko initialize karta hai		
9	Inheritance	Current B object ko represent karta hai	Parent A ki implementation tak pahunchta hai		
10	Example	this.marks = 90	super(name, age)		
11	Private member	Current class ke accessible #private members access kar sakta hai	Parent ke #private members ko bypass nahi kar sakta		
12	Simple meaning	"Mera/current object"	"Mere parent ka"		

2. JavaScript me destructor hota hai?

✗ Traditional destructor nahi hota.

JavaScript Garbage Collector (GC) use karta hai.

```
</> JavaScript

class A {
  constructor() {
    console.log("Constructor");
  }
}

let a = new A();

a = null;
```

Jab object unreachable ho jata hai:

a → Object



Jab object unreachable ho jata hai:

a —————> Object

phir:

```
</> JavaScript
```

```
a = null;
```

to:

a —————> null

Object

↓

unreachable

↓

Garbage Collector

↓

memory cleanup

```
</> JavaScript

class A {

    // Private method
    #A_ka_private(a, b) {
        console.log("A ka private function:", a + b);
    }

    // Public method
    A_ka_public(name, age, ispass) {
        console.log(name, age, ispass);
    }

    // Only ONE constructor
    constructor(name, age, ispass, marks) {

        console.log("A constructor:");

        console.log(name, age, ispass, marks);

        this.city = "Nalkheda";
        this.grade = "A";
        this.salary = 5.44;
```



```
        this.city = "Nalkheda";
        this.grade = "A";
        this.salary = 5.44;

        this.name = "Shivam";
        this.marks = 43.44;
    }
}

class B extends A {

    // Private method
    #B_ka_private(a, b) {
        console.log("Decimal power:", a ** b);
    }

    // Public method
    B_ka_public(a, b, c) {
        console.log(a * b * c);
    }
}
```



</> JavaScript

```
// B constructor
constructor(name, age, ispass, marks) {

    // FIRST call parent constructor
    super(name, age, ispass, marks);

    // AFTER super(), use this
    this.ispass = true;
    this.age = 54;

    console.log("B constructor");
}

work() {

    // Parent A ka public method
    super.A_ka_public("Tarun", 22, true);
}
}

let b = new B("Tarun", 44, true, 4.4);
```



```
</> JavaScript
```

```
b.work();
```

Output

```
A constructor:  
Tarun 44 true 4.4
```

```
B constructor
```

```
Tarun 22 true
```

23a0c22cc23c

Ab pura flow samjho

Jab tum:

```
</> JavaScript
```

```
let b = new B("Tarun", 44, true, 4.4);
```

likhte ho:

Step 1

B ka constructor call hota hai:

```
</> JavaScript
```

```
constructor(name, age, ispass, marks)
```

Step 2

B ke constructor me:

```
</> JavaScript  
super(name, age, ispass, marks);
```

milta hai.

Iska matlab:

A ka constructor call karo.

Step 3

A ka constructor execute hota hai:

Step 3

A ka constructor execute hota hai:

```
</> JavaScript  
constructor(name, age, ispass, marks) {  
  
    this.city = "Nalkheda";  
    this.grade = "A";  
    this.salary = 5.44;  
  
    this.name = "Shivam";  
    this.marks = 43.44;  
}
```

Step 4

A constructor complete hone ke baad B me wapas aate hain.

Step 4

A constructor complete hone ke baad B me wapas aate hain.

Ab:

```
</> JavaScript
```

```
this.ispass = true;  
this.age = 54;
```

execute hoga.

1. Single Inheritance

Ek child → ek parent

 JavaScript

```
class A {  
  showA() {  
    console.log("A");  
  }  
}  
  
class B extends A {  
  showB() {  
    console.log("B");  
  }  
}  
  
let b = new B();  
  
b.showA(); // A  
b.showB(); // B
```



2. Multilevel Inheritance

Grandparent → Parent → Child

`</>` JavaScript

```
class A {  
  showA() {  
    console.log("A");  
  }  
}  
  
class B extends A {  
  showB() {  
    console.log("B");  
  }  
}  
  
class C extends B {  
  showC() {  
    console.log("C");  
  }  
}
```



```
let c = new C();
```

```
c.showA();
```

```
c.showB();
```

```
c.showC();
```

Diagram:

A

↓

B

↓

C

C ko:

A ke members

+

B ke members

+

C ke members

3. Hierarchical Inheritance

Ek parent → multiple children

```
</> JavaScript

class A {
  showA() {
    console.log("A");
  }
}

class B extends A {
  showB() {
    console.log("B");
  }
}

class C extends A {
  showC() {
    console.log("C");
  }
}
```



5. Lekin JavaScript me Multiple Inheritance jaisa behavior achieve kar sakte hain

JavaScript me **Mixins** use kar sakte ho.

```
</> JavaScript

const A = {
  showA() {
    console.log("A");
  }
};

const B = {
  showB() {
    console.log("B");
  }
};

class C {}

Object.assign(C.prototype, A, B);
```



```
</> JavaScript
```

```
class C {}
```

```
Object.assign(C.prototype, A, B);
```

```
let c = new C();
```

```
c.showA();
```

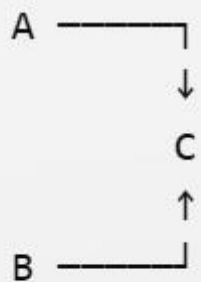
```
c.showB();
```

Output:

A

B

Diagram:



Js oops

JavaScript

```
class C extends A, B {  
}
```

to:

JavaScript

```
let c = new C();  
  
c.show();
```

? Kaunsa chale?

```
A.show()  
    OR  
B.show()
```

Ye ambiguity create karega.

Isliye JavaScript ka `extends` syntax sirf ek direct parent class allow karta hai.

Hinglish me:



JavaScript me multiple inheritance directly nahi hota kyunki agar ek child ke do parents hon aur dono me same method/property ho, to conflict ho sakta hai ki child kis parent wala member use kare. Diamond problem bhi ho sakta hai. Isliye JS `extends` me ek hi direct parent allow karta hai. Multiple functionality ke liye mixins/composition use kar sakte hain.

Ek line me yaad rakho:

Multiple Inheritance



2 Parents



Same method/property?



Ambiguity ⚠️



Diamond Problem 💎



JS: Direct multiple class inheritance ❌



Mixins / Composition ✅



3. JavaScript me method overloading?

Yahan important distinction hai.

Traditional method overloading ❌

JavaScript me C++/Java jaisa:

```
add(int, int)
add(int, int, int)
add(float, float)
```

same class me genuinely overloaded methods **nahi hote**.

Example:

```
class A {
  add(a, b) {
    console.log(a + b);
  }
  add(a, b, c) {
    console.log(a + b + c);
  }
}
```

JavaScript me second `add()` effectively first ko overwrite kar dega.

```
</> JavaScript
let a = new A();
a.add(2, 3);
```

Traditional overloading jaisa behavior nahi hoga.

4. Lekin JS me overloading jaisa behavior bana sakte

Default parameters / rest parameters se:

</> JavaScript

```
class A {  
  
  add(...numbers) {  
    let sum = 0;  
  
    for (let n of numbers) {  
      sum += n;  
    }  
  
    console.log(sum);  
  }  
}  
  
let a = new A();  
  
a.add(2, 3);  
a.add(2, 3, 4);  
a.add(2, 3, 4, 5);
```



5. JavaScript me polymorphism ke important forms

Type	JavaScript me?	Example
Method Overriding	✓	Child parent method ko redefine kare
Runtime polymorphism	✓	<code>obj.method()</code> based on actual object
Duck typing	✓	Object me required method ho to use kar sakte hain
Method Overloading	✗ Traditional	Same method name + different signatures
Overloading-like behavior	✓	Rest/default parameters
Operator overloading	✗ Traditional	JS me user-defined operator overload nahi

1. Runtime Polymorphism in JavaScript

Definition

Runtime polymorphism me method ka actual implementation runtime par decide hota hai, based on actual object jo method ko call kar raha hai.

Simple Hinglish:

Method ka naam same hota hai, lekin runtime par object decide karta hai ki kaunsa version chalega.

```
class A {  
  
    work() {  
        console.log("A ka work");  
    }  
}  
  
class B extends A {  
  
    work() {  
        console.log("B ka work");  
    }  
}  
  
class C extends A {  
  
    work() {  
        console.log("C ka work");  
    }  
}
```

.1.

5-29a0c22ee23e

</> JavaScript

```
let obj;
```

```
obj = new B();  
obj.work();
```

```
obj = new C();  
obj.work();
```

```
obj = new A();  
obj.work();
```

Output:

B ka work

C ka work

A ka work

```
</> JavaScript
```

```
obj
```

same hai.

Method:

```
</> JavaScript
```

```
obj.work()
```

same hai.

Lekin actual object change ho raha hai:

```
obj = new B()
```

↓

```
B ka work()
```

```
obj = new C()
```

2. Parent class reference wala example

[Upgr](#)

Java/C++ me tumne ye concept dekha hoga:

```
Animal *a = new Dog();
```



JavaScript me exact pointer/reference declaration nahi hoti, but behavior similar ho sakta hai.

JavaScript



```
class Animal {  
    sound() {  
        console.log("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    sound() {  
        console.log("Bhow Bhow");  
    }  
}
```



JavaScript

```
class Cat extends Animal {  
  
    sound() {  
        console.log("Meow");  
    }  
}
```

Function:

JavaScript

```
function makeSound(animal) {  
    animal.sound();  
}
```

Ab:

JavaScript

```
makeSound(new Dog());  
makeSound(new Cat());  
makeSound(new Animal());
```



Runtime par decide hota hai:

```
new Dog()  
  ↓  
Dog.sound()  
  
new Cat()  
  ↓  
Cat.sound()  
  
new Animal()  
  ↓  
Animal.sound()
```

👉 Runtime Polymorphism

"Does JavaScript support abstract classes?"

✦ Upgrad

Best answer:

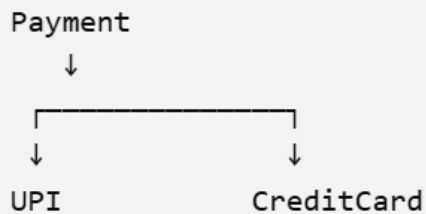
JavaScript does not have a built-in `abstract` keyword or native abstract classes like Java or C++. However, we can simulate abstract-class behavior using a normal class, `new.target` to prevent direct instantiation, and methods that throw errors to require child classes to implement them.

Hinglish:

JS me built-in abstract class nahi hoti. `abstract` keyword bhi nahi hota. Lekin normal class + `new.target` + error throwing method ka use karke abstract class jaisa behavior create kar sakte hain.

9. Abstract class ka real purpose

Maan lo tum ek software bana rahe ho jisme:



Har payment ko:

`pay()`

method implement karna hi padega.

Abstract-like class:

Abstract-like class:

```
</> JavaScript

class Payment {

  constructor() {

    if (new.target === Payment) {
      throw new Error("Payment is abstract");
    }
  }

  pay() {
    throw new Error("Child must implement pay()");
  }
}
```

UPI:

UPI:

```
</> JavaScript  
  
class UPI extends Payment {  
  
    pay() {  
        console.log("Payment through UPI");  
    }  
}
```

Credit Card:

```
</> JavaScript  
  
class CreditCard extends Payment {  
  
    pay() {  
        console.log("Payment through Credit Card");  
    }  
}
```



```
</> JavaScript

let u = new UPI();
let c = new CreditCard();

u.pay();
c.pay();
```

Output:

```
Payment through UPI
Payment through Credit Card
```

"Does JavaScript support abstract classes?"

✦ Upgrad

Best answer:

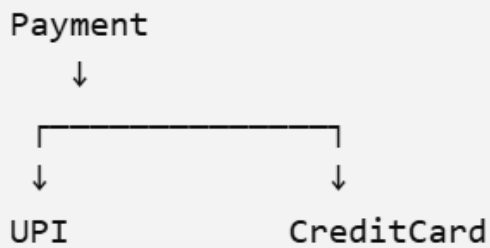
JavaScript does not have a built-in `abstract` keyword or native abstract classes like Java or C++. However, we can simulate abstract-class behavior using a normal class, `new.target` to prevent direct instantiation, and methods that throw errors to require child classes to implement them.

Hinglish:

JS me built-in abstract class nahi hoti. `abstract` keyword bhi nahi hota. Lekin normal class + `new.target` + error throwing method ka use karke abstract class jaisa behavior create kar sakte hain.

9. Abstract class ka real purpose

Maan lo tum ek software bana rahe ho jisme:



Har payment ko:

`pay()`

method implement karna hi padega.

Abstract-like class:

Abstract-like class:

```
</> JavaScript  
  
class Payment {  
  
    constructor() {  
  
        if (new.target === Payment) {  
            throw new Error("Payment is abstract");  
        }  
    }  
  
    pay() {  
        throw new Error("Child must implement pay()");  
    }  
}
```

UPI:

UPI:

```
</> JavaScript

class UPI extends Payment {

    pay() {
        console.log("Payment through UPI");
    }
}
```

Credit Card:

```
</> JavaScript

class CreditCard extends Payment {

    pay() {
        console.log("Payment through Credit Card");
    }
}
```

↓

```
</> JavaScript

let u = new UPI();
let c = new CreditCard();

u.pay();
c.pay();
```

Output:

```
Payment through UPI
Payment through Credit Card
```

Static ka meaning samjho.

[Upgrade](#)

1. Static property kya hoti hai?

Definition

Static property wo property hoti hai jo class se belong karti hai, kisi individual object se nahi.

Simple Hinglish:

```

Normal property
  ↓
Object ki property

Static property
  ↓
Class ki property
  
```



2. Normal property vs Static property

```

</> JavaScript

class Student {

  constructor(name) {
    this.name = name;
  }

  static college = "Medi-Caps";
}
  
```

Object:

```

</> JavaScript

let s1 = new Student("Tarun");
let s2 = new Student("Shivam");
  
```

Normal property:

```
</> JavaScript  
  
console.log(s1.name);  
console.log(s2.name);
```

Output:

```
Tarun  
Shivam
```

name object-specific hai.

Lekin static:

```
</> JavaScript  
  
console.log(Student.college);
```



Output:

Medi-Caps

Ye nahi:

```
</> JavaScript  
console.log(s1.college); // ❌
```

Static property ko normally **class name** se access karte hain:

```
</> JavaScript  
Student.college
```

3. Static property ka simple example

```
</> JavaScript

class Student {

    static college = "Medi-Caps";

    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
}
```

Objects:

```
</> JavaScript

let s1 = new Student("Tarun", 22);
let s2 = new Student("Shivam", 23);
```

Normal:

```
</> JavaScript  
  
console.log(s1.name);  
console.log(s2.name);
```

Static:

```
</> JavaScript  
  
console.log(Student.college);
```

Output:

```
Tarun  
Shivam  
Medi-Caps
```

4. Static property sab objects ke liye common hoti hai?

Conceptually haan, value class-level/common hoti hai, but technically wo objects ki own property nahi hoti.

```
Student
|
static college
|
"Medi-Caps"
```



Objects:

```
s1 → name = Tarun
s2 → name = Shivam
s3 → name = Rahul
```



Sab students same class-level information use kar sakte hain:



 JavaScript



5. Static variable ko modify kar sakte hain

```
</> JavaScript  
  
class Student {  
    static college = "Medi-Caps";  
}  
  
console.log(Student.college);  
  
Student.college = "IIT";  
  
console.log(Student.college);
```

Output:

Medi-Caps
IIT



8. Static vs Instance Property

Point	Instance Property	Static Property
Belongs to	Object	Class
Access	<code>obj.name</code>	<code>Class.name</code>
Har object ki separate copy	Usually 	
Common class-level value		
Constructor me	<code>this.x</code>	<code>Class.x</code>
Example	<code>this.name</code>	<code>Student.count</code>